

Documentação Técnica — PrMO (Prompt Management Office)

Plataforma de **governança de IA** e **repositório de prompts/propriedade intelectual** da Vanguarda Martech. Este documento consolida arquitetura, requisitos, regras de negócio, modelo de dados, API, segurança e implantação.

Documentos relacionados: [GOVERNANCA](#) · [REGRAS-DE-NEGOCIO](#) · [SUPABASE](#) · normas em [normas/](#) · histórico em [DIARIO-SESSAO](#).

- Versão do documento:** 1.0 · **Última atualização:** 2026-08-12
- Status do sistema:** em produção (backend Render + frontend GitHub Pages + banco Supabase)

1. Visão geral

O PrMO centraliza a governança do uso de IA na agência: **biblioteca de prompts** versionada (padrão NIA-001), **stack de ferramentas homologadas**, **indicadores executivos**, **compliance/riscos**, **trilha de auditoria** e **observabilidade** — alimentados por uma base de diagnóstico de 30 dias (anonimizada, LGPD).

Perfis de uso: - **Colaborador (User):** consulta a Biblioteca de prompts e propõe novos (entram como *Revisão pendente*). - **Gestor (Manager) / Administrador (Admin):** homologam prompts, gerenciam stack, veem painéis, auditoria e integrações.

2. Arquitetura

```
Navegador (usuário)
| HTML/CSS/JS estático (SPA de página única)
▼
GitHub Pages —fetch (JWT Bearer)→ FastAPI (Render: prmo-api)
| SQLAlchemy 2 + Pydantic 2
▼
Supabase / PostgreSQL 17 (RLS)
```

CAMADA	TECNOLOGIA	HOSPEDAGEM
Frontend	HTML + CSS + JS puro (single-file, sem build), libs de export via CDN sob demanda	GitHub Pages
Backend	Python 3.11, FastAPI, SQLAlchemy 2.0, Pydantic 2, JWT (python-jose), passlib/bcrypt	Render (prmo-api)
Banco	PostgreSQL 17 (Supabase); SQLite em dev	Supabase (prmo-governanca , sa-east-1)
Observabilidade	structlog (logs JSON) + Prometheus client	no serviço
CI/CD	GitHub Actions (deploy Pages) + Render (auto-deploy no push)	GitHub / Render

Princípios: **acesso ao banco mediado exclusivamente pelo backend** (nenhum cliente fala direto com o Supabase); **agnóstico de banco** via `DATABASE_URL`; **dados anonimizados** enquanto o repositório for público.

3. Requisitos

3.1 Requisitos funcionais (RF)

- **RF01 — Autenticação:** login por e-mail/senha com JWT (access + refresh).
- **RF02 — Autocadastro corporativo:** autocadastro público restrito a e-mails `@vanguardamartech.com.br`; novo usuário entra como **User**.
- **RF03 — RBAC:** User vê apenas a Biblioteca (leitura + propor); Manager/Admin acessam todos os módulos.
- **RF04 — Biblioteca de prompts:** listar, cadastrar, **ver/editar** e **homologar** (Aprovar/Reprovar) prompts no padrão NIA-001.
- **RF05 — Stack & homologação:** cadastro e status de ferramentas (Homologada/Em análise/Restrita/Reprovada).
- **RF06 — Indicadores executivos:** KPIs e listas calculados da base migrada (colaboradores, ativos, riscos, gaps, oportunidades, adoção, stack).
- **RF07 — Ranking de setores por ativos digitais:** macroáreas ordenadas por nº de ativos; linhas **expansíveis** com os ativos do setor.
- **RF08 — Compliance & auditoria:** ocorrências, riscos declarados, exposição e **trilha de auditoria** (`audit_logs`).
- **RF09 — Teto de custo:** política global de custo por chamada; prompts acima do teto são sinalizados.
- **RF10 — Registros (30d):** consulta aos registros migrados por tipo (asset, risk, knowledge, opportunity, diagnostic, plan30).
- **RF11 — Observabilidade:** healthchecks e métricas do sistema/painel admin.
- **RF12 — Integrações:** conectores configuráveis (RD Station, ICLIPS, VJOB, **VanguardIA**) com test/sync.
- **RF13 — Exportação de relatório:** exportar o painel executivo em **PDF, XLSX, PNG, JPEG**.

3.2 Requisitos não funcionais (RNF)

- **RNF01 — Segurança:** senhas com bcrypt; JWT assinado (HS256); RLS deny-by-default no banco; CORS controlado.
 - **RNF02 — LGPD:** repositório público só contém dados **anonimizados**; dados sensíveis/PII só em infraestrutura privada (Supabase).
 - **RNF03 — Portabilidade:** backend agnóstico de banco (SQLite/PostgreSQL) via `DATABASE_URL`.
 - **RNF04 — Resiliência:** `pool_pre_ping` + `pool_recycle` para conexões atrás do pooler; seeds **idempotentes**.
 - **RNF05 — Observabilidade:** logs estruturados JSON e métricas Prometheus.
 - **RNF06 — Auditabilidade:** toda criação/edição/homologação e violação de regra é registrada.
 - **RNF07 — Acessibilidade/UX:** navegação por teclado nos controles principais; `prefers-reduced-motion`; feedback de carregamento.
-

4. Modelo de dados (PostgreSQL)

DDL autoritativo em `../db/supabase_schema.sql`. Principais tabelas:

TABELA	DESCRIÇÃO
users	Usuários (papel <code>userrole</code> : Admin/Manager/User; status <code>userstatus</code>)
activities , audit_logs	Atividades e trilha de auditoria
tools , skills , prompts	CRUD base (protótipo original)
integration_configs , integration_sync_logs	Integrações e logs de sincronização
gov_clients	Portfólio/contas
gov_stack_tools	Stack de ferramentas + status de homologação
gov_prompts	Biblioteca de prompts (padrão NIA-001 — ver §6)
gov_skills , gov_training , gov_adoption	Pessoas, trilhas, adoção
gov_initiatives , gov_incidents	Iniciativas e ocorrências
gov_cost_policy	Teto de custo por chamada
gov_registry	Registros migrados (30d) anonimizados: <code>registry</code> , <code>code</code> , <code>data</code> (JSON)

`gov_prompts` (campos NIA-001): `code` (`PROMPT-ÁREA-NNN`), `version` , `ptype` (A–E), `tool` , `author` , `data_class` , além de `title` , `description` , `area` , `control` (Aprovado/Revisão pendente/Reprovado), `content` , `repo_url` , `cost_per_call` , `last_review` , `uses` .

5. API (rotas principais)

Base do backend: `https://prmo-api.onrender.com` . Autenticação: `Authorization: Bearer <access_token>` .

Autenticação (`/auth`)

MÉTODO	ROTA	ACESSO	DESCRIÇÃO
POST	<code>/auth/login</code>	público	Login → tokens + usuário
POST	<code>/auth/signup</code>	público	Autocadastro restrito a <code>@vanguardamartech.com.br</code> (papel User)
POST	<code>/auth/refresh</code>	público	Renova access token
GET	<code>/auth/me</code> · <code>/auth/check</code>	autenticado	Sessão atual
POST	<code>/auth/change-password</code>	autenticado	Alterar senha

Governança (/governance)

MÉTODO	ROTA	ACESSO	DESCRIÇÃO
GET	/overview	autenticado	KPIs, compliance, adoção, stack, ranking_setores, custo
GET/POST	/prompts	GET auth / POST auth	Listar / cadastrar prompt (User → Revisão pendente)
PUT	/prompts/{id}	Manager	Editar prompt (regras R1/R4 + auditoria)
POST	/prompts/{id}/homologar	Manager	Aprovar/Reprovar (regras R1–R4)
GET	/rules	autenticado	Catálogo das regras de negócio (R1–R4)
GET/PUT	/cost-policy	GET auth / PUT Manager	Teto de custo
GET/POST	/clients /stack /skills /initiatives /incidents	POST Manager	Cadastros de governança
GET	/training /adoption /audit /observability	autenticado/Manager	Listas e painéis
GET	/registry · /registry/{name}	Manager	Resumo e detalhe dos registros migrados

Admin (/admin) e Integrações (/integrations)

- /admin/users , /admin/audit-logs , /admin/metrics , /admin/dashboard , /admin/integrations (Admin).
- /integrations , /integrations/{name} (GET/PUT), /integrations/{name}/test , /integrations/{name}/sync , /integrations/{name}/logs .

Saúde/observabilidade

- /health/live , /health/ready , /health/startup , /health/metrics .

6. Regras de negócio

Detalhe e base normativa em REGRAS-DE-NEGOCIO (mapeia as normas em normas/). Consultáveis via GET /governance/rules .

6.1 Padrão NIA-001 do prompt

- Todo prompt carrega metadados obrigatórios: código (PROMPT-ÁREA-NNN , auto-gerado), versão, tipo (A Operacional · B Analítico · C Estratégico · D Automação · E Criativo, auto-classificado), ferramenta, autor, classificação da informação.
- Sigla de área derivada das macroáreas do Manual (CRI, ATD, PLA, MID, SOC, CML, INB, RH, FIN, DEV, CEO...).

6.2 Motor de regras (aplicado no ciclo do prompt)

ID	BASE	MOMENTO	REGRA
R1	NIA-001 §13 / Política §6	Cadastro/Edição	Bloqueia credenciais, segredos ou PII (senha, token, <code>api_key</code> , CPF, cartão, termos de fraude).
R2	Política §4	Homologação	Só aprova prompt com ferramenta homologada no stack.
R3	NIA-001 §5/§12	Homologação	Aprovação exige metadados mínimos: código + objetivo/descrição + conteúdo.
R4	Política §7	Cadastro/Edição/Homologação	Dado Restrito não pode apontar para repositório público externo.

- Violações → **HTTP 422** com `{"regras":[...]}` e registro em `audit_logs`. **Reprovar** nunca é bloqueado.

6.3 Outras regras

- **Autocadastro (RF02)**: somente domínio `@vanguardamartech.com.br`; papel inicial **User**.
- **RBAC (RF03)**: User só acessa a Biblioteca; homologação é de Manager/Admin.
- **Fluxo do prompt (NIA-001 §10)**: Criação → Teste → Validação → Homologação → Publicação → Monitoramento → Melhoria contínua.
- **Teto de custo (RF09)**: política global (`gov_cost_policy`); prompts com `cost_per_call` acima do teto são sinalizados.
- **Ranking de setores (RF07)**: macroáreas ordenadas pelo nº de ativos na base de conhecimento.

7. Segurança e conformidade

- **Autenticação**: JWT HS256 (access + refresh); senhas com bcrypt (passlib).
- **RBAC**: dependências `get_current_user`, `get_current_manager_user`, `get_current_admin_user`.
- **Banco (RLS)**: RLS **habilitado em todas as tabelas** do schema `public` sem policies (deny-by-default); a Data API (`anon` / `publishable`) não lê nem escreve; o backend conecta como papel `postgres` e ignora RLS.
- **LGPD**: repositório público contém apenas dados anonimizados (dicionário de redação aplicado no seed); carga completa/PII apenas no Supabase privado.
- **Segredos**: `DATABASE_URL` / `SECRET_KEY` fora do repositório (Render `sync:false`, cofre).
- **Auditoria**: ações relevantes e violações de regra em `audit_logs` (`GET /governance/audit`).

8. Implantação e configuração

Backend (Render — `prmo-api`)

- Build: `pip install -r backend/requirements.txt`; start: `uvicorn main:app` (`render.yaml`).
- Variáveis: `DATABASE_URL` (`sync:false` — Session pooler do Supabase, ver `SUPABASE`), `SECRET_KEY` (gerada), `APP_NAME`, `DEBUG=False`, `LOG_LEVEL`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`.
- No boot: `Base.metadata.create_all` + seeds idempotentes (usuários, registros 30d, 52 candidatos NIA-001, dados curados).

Frontend (GitHub Pages)

- Site estático em `frontend/index.html`; workflow `deploy-frontend-pages.yml` publica sem build.
- Constante `API` aponta para `https://prmo-api.onrender.com`.

Banco (Supabase)

- Schema + dados migrados via MCP; RLS habilitado. Passo a passo em `SUPABASE`.

9. Integrações

Conectores no padrão `integrations/` (base configurável por `base_url` + `api_key`): **RD Station**, **ICLIPS**, **VJOB** e **VanguardIA**. Endpoints `test` / `sync` por integração. O `sync` do **VanguardIA** importa agentes/prompts homologados para a Biblioteca como candidatos (*Revisão pendente*) já no padrão NIA-001.

10. Limitações conhecidas e pendências

- **Cold start (Render free tier)**: ~50s na 1ª requisição após ociosidade. Mitigado com overlay + ping de aquecimento; solução definitiva = pinger externo ou upgrade de plano.
- **VanguardIA**: conector pronto e **desabilitado** — falta `base_url` + token reais.
- **SSO Google Workspace**: planejado (login com trava `hd=vanguardamartech.com.br`) — requer OAuth Client ID/Secret.
- `/governance/overview` hoje é acessível a qualquer autenticado (o recorte por perfil do dashboard é aplicado no frontend).
- **Autocadastro** confere o domínio, mas não verifica posse do e-mail (mitigado pelo SSO quando ativado).
- Exportações usam bibliotecas via CDN carregadas no navegador do usuário.

11. Estrutura do repositório (resumo)

```
backend/          API FastAPI (governance.py, auth/, admin/, integrations/, models.py, crud.py, config.py)
backend/data/     registros_30dias.json (anonimizado)
frontend/index.html SPA estática
db/supabase_schema.sql DDL autoritativo
docs/            GOVERNANCA, REGRAS-DE-NEGOCIO, SUPABASE, normas/, DIARIO-SESSAO, esta documentação
render.yaml      blueprint do backend
.github/workflows/ CI + deploy Pages
```